

Clustering

Overview

- unsupervised machine learning
- identify salient patterns in the data
- k-means
- lossy compression scheme
- convergence and initialization
- hierarchical clustering

Reading

Pattern Recognition and Machine Learning

9.1 K-Means Clustering

<https://oa.ee.tsinghua.edu.cn/ouzhijian/pgm/pgm-pdf/PRML%20book.pdf>

Elements of Statistical Learning

14.3 Cluster Analysis

<https://jqichina.wordpress.com/wp-content/uploads/2012/02/the-elements-of-statistical-learning.pdf>

PRML: K-Means Clustering

K is assumed to be a known value

9.1 K-Means Clustering

What

K-Means Means..

- a non-probabilistic clustering method
- divides N datapoint into D – *dimensional* Euclidean space into K clusters
- each cluster is represented by a prototype vector μ_k (cluster center)
- objective is to assign each point to a cluster so that points are as close as possible to their assigned center

Key Representation	- for each datapoint x_n there exist binary indicator variables r_{nk} $r_{nk} = 1$: if x_n is assigned to cluster k ← 1-of-K coding $r_{nk} = 0$: else - using these indicators, K-means minimizes the distortion objective sum of squared distances from every point to its assigned cluster center → $J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} x_n - \mu_k ^2$
How the Algorithm Works	K-means alternates between two steps assignment step - keep the centers μ_k fixed - assign each point to the nearest center - formally, set $r_{nk} = 1$ for the cluster whose center gives the smallest square distance to x_n update step - keep the assignments fixed - recompute each center as the mean of the points assigned to that cluster $\mu_k = \frac{\sum_n r_{nk} x_n}{\sum_n r_{nk}}$ each prototype becomes the mean of its cluster members
Iteration and Convergence	- these two steps cycle until - the assignments stabilize - exit criteria are met - each step reduces the objective J - only guaranteed to reach a local minimum

Relation to Expectation Maximization (EM)	- these two <i>K – means</i> phases correspond to the <i>E – step</i> and <i>M – step</i> <i>E – step</i> ↔ reassigning points <i>M – step</i> ↔ recompute centers <i>K – means</i> is a special non-probabilistic limit of <i>EM</i> for Gaussian mixtures
Practical Behavior	“The Old Faithful” example shows - the algorithm depends on the initial choice of centers poor initialization can require more iterations - a common practical initialization is to choose <i>K</i> random data points as the initial centers - the cost function <i>J</i> decreases after successive <i>E</i> and <i>M</i> steps until convergence
Computational Issues	- a direct implementation can be slow because each assignment step compares every data point with every prototype vector - the text mentions speedups based on tree-based data structures for nearby points triangle-inequality tricks to avoid unnecessary distance computations
Online Stochastic Version	updates the nearest prototype after each incoming point decreases over time ↓ $\mu_k^{new} = \mu_k^{old} + \eta_n(x_n - \mu_k^{old})$ - where η_n is a learning rate - makes <i>K – means</i> suitable for sequential or streaming-style updates

Limits of Standard K- Means	• standard <i>K – means</i> uses squared Euclidean distance • creates two important limitations ○ it is not appropriate for some variable types, such as categorical labels ○ the mean can be sensitive to outliers, so the cluster centers may be non-robust
Generalization: K-medoids	• generalizes <i>K – means</i> by replacing squared Euclidean distance with a more general dissimilarity measure $V(x, x')$, yielding a new objective:$J' = \sum_{n=1}^N \sum_{k=1}^K r_{nk} V(x_n, \mu_k)$ • this leads to <i>K – medoids</i> ○ assignment still sends each point to the nearest prototype under the chosen dissimilarity ○ the update step is harder ○ the prototype is often restricted to be one of the actual data points in the cluster

Hard Assignments vs Soft Assignments	• hard assignment ○ each point is assigned to exactly one cluster ○ some points may lie roughly between centers ○ so, hard assignment may be artificial • soft assignments ○ probabilistic model reflects uncertainty
9.1.1 Image Segmentation and Compression	
Image Segmentation Example	• the textbook example applies K – means to color images by treating each pixel as a data point in a $3 – D$ RGB space ○ one coordinate for ▪ red intensity ▪ green intensity ▪ blue intensity • K – means ○ groups similar colors together ○ redraws each pixel using the color of its assigned cluster center ○ result is an image represented using only K colors
Important Caveat for Example	• a major weakness is that it ignores spatial proximity • two pixels far apart but similar in color may be put in the same cluster • nearby pixels are not treated as related just because they are adjacent in the image • this approach is simple, but not strong enough for general image segmentation

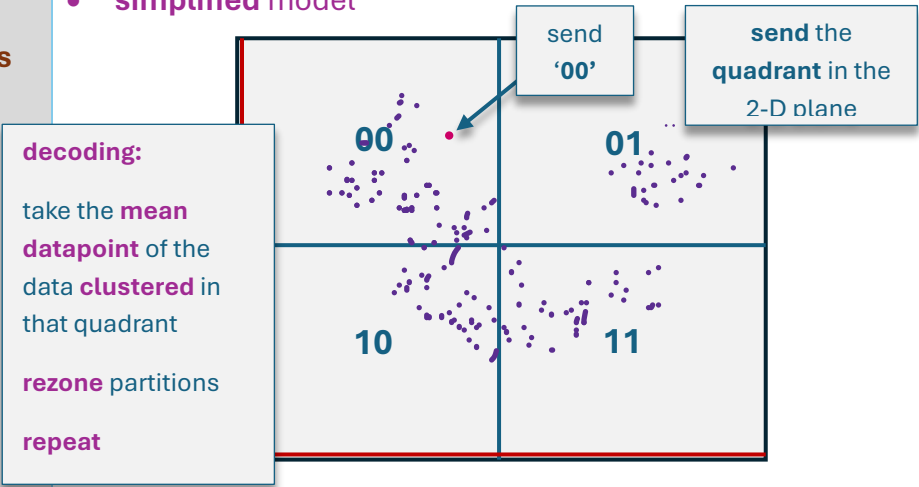
Compression Viewpoint	- the same setup also gives a form of lossy compression - instead of storing every original pixel vector - store only the index k of the nearest cluster center - also, store the K center vectors themselves - this is called vector quantization cluster centers are called code-book vectors approximation is lossy because each original pixel is replaced by the nearest center, not preserved exactly
Bit-Cost Calculation	- if an image has N pixels and each RGB pixel uses 24 bits total - original image cost = $24N$ bits compressed image cost = $24K + N \log_2 K$ bits - for the image in the example - the original requires 1,036,800 bits K-means-compressed versions require 43,248 bits for $K = 2$ 86,472 bits for $K = 3$ 173,040 bits for $K = 10$ - these are 4.2%, 8.3%, and 16.7% of the original size, respectively - the trade-off is clear, smaller K gives more compression - smaller worse visual quality

rounded up
as needed

Summary	
Takeaway from Image Example	• shows that K-means ○ clusters similar vectors ○ can reduce data to a small codebook ○ creates a direct trade-off between compression and fidelity • the text notes that for better image compression ○ cluster small blocks of neighboring pixels
Overall Takeaways for K-means	• K-means as an iterative algorithm that alternates between ○ assigning each point to its nearest center ○ updating each center to the mean of its assigned points • properties of <i>K – means</i> ○ minimizes within-cluster squared distance ○ converges to a local minimum ○ uses hard assignments ○ can be extended beyond Euclidean distance ○ can be applied to tasks like image segmentation and lossy compression through vector quantization ○ simplicity is its strength ○ but is limited by assumptions

exploits local correlations in natural images

Async	
Clustering Goals	
Two Classes of Learning Problems	• supervised learning ○ learning from pre-labeled data ▪ class labels (in classification) ▪ output values (regression) ○ train data: (X, Y) for inputs X and labels Y • unsupervised learning ○ learning from unlabeled, unannotated data ○ train data: X for unlabeled data • learn from salient patterns from the input features themselves
Unsupervised Learning Methods	• clustering methods ○ identify data groupings ○ non-probabilistic method ▪ hierarchical clustering ▪ K-means algorithm ○ probabilistic method ▪ mixture model • density estimation ○ determine data distribution • dimensionality reduction ○ determine low-dimensional representation

Clustering	• organizes data into clusters such that there is ○ high intra-cluster similarity ○ low inter-cluster similarity • optimal for finding natural groupings among objects
Clustering as Lossy Compression	
Data Shape	• data is stored as different data types with each having different sizes in memory • even within the same data type group – such as float – there are variations such as different degrees of precision
The Problem	• suppose you transmit the coordinates of points drawn randomly from this dataset • you can install decoding software at the receiver • you're only allowed to send two bits per point • this requires lossy compression • loss is defined as the sum-squared error between decoded coordinates and original coordinates
Solutions: Minimizes Loss	• simplified model  decoding: take the mean datapoint of the data clustered in that quadrant rezone partitions repeat

Optimizing Partitions and Centroids

K-Means

- ask user **how many clusters**
- randomly **guess** the k cluster **center locations**
- each datapoint **determines** which **center** is closest
- each **center** “**owns**” a set of **datapoints**
- each center **determines center** of “owned” points
- **center relocates** to new position
- **repeat** until terminated

K-Means Questions

- **what** is it **trying to optimize**?
- are we **sure** it **will terminate**?
- are we **sure** it will **find an optimal clustering**?
- **how** should we **start** it?
- **how** could we **automatically choose** the **number of centers**?

K-Means Loss

Distortion

- **encoder** function

$$\mathbb{R}^m \rightarrow \{1, \dots, k\}$$

- **decoder** function

$$\{1, \dots, k\} \rightarrow \mathbb{R}^m$$

- **define**

$$\text{Distortion} = \sum_{i=1}^R \left(x_i - \text{DECODE}(\text{ENCODE}(x_i)) \right)^2$$

- if we set,

$$\text{DECODE}[j] = c_j \quad \text{then,}$$

$$\text{Distortion} = \sum_{i=1}^R \left(x_i - c_{\text{ENCODE}(x_i)} \right)^2$$

Minimizing Loss With Respect to Partitions

The Minimal Distortion

$$\text{Distortion} = \sum_{i=1}^R (x_i - c_{\text{ENCODE}(x_i)})^2$$

- what **priorities** must **cluster assignments** have when **distortions** **minimized**?
 - x_i must be **encoded** by its **nearest center**

$$c_{\text{ENCODE}(x_i)} = \underset{c_j \in \{c_1, c_2, \dots, c_k\}}{\text{argmin}} (x_i - c_j)^2$$

at the **minimal distortion**

Minimizing Loss With Respect to Centroids

The Minimal Distortion (2)

- what **properties** must **centers** $c_1, c_2, \dots, c_k$ have when **distortion** is **minimized**?
- the **partial derivative** of **distortion** with respect to each **center location** must be **zero**
- this is a **convex problem**, so the **minimum** is where the **derivative equals zero**

Steps

- from above: the **partial derivative** of **distortion** with respect to each **center location** must be **zero**

$$\text{Distortion} = \sum_{i=1}^R (x_i - c_{\text{ENCODE}(x_i)})^2$$

$$= \sum_{j=1}^k \sum_{i \in \text{OwnedBy}(c_j)} (x_i - c_j)^2$$

OwnedBy(c_j):
the set of records
owned by center c_j

$$\begin{aligned}
 \frac{\partial \text{Distortion}}{\partial c_j} &= \frac{\partial}{\partial c_j} \sum_{i \in \text{OwnedBy}(c_j)} (x_i - c_j)^2 \\
 &= -2 \sum_{i \in \text{OwnedBy}(c_j)} (x_i - c_j) \\
 &= 0 \text{ (for a minimum)}
 \end{aligned}$$

Alternating and Convergence

At the Minimum Distortion

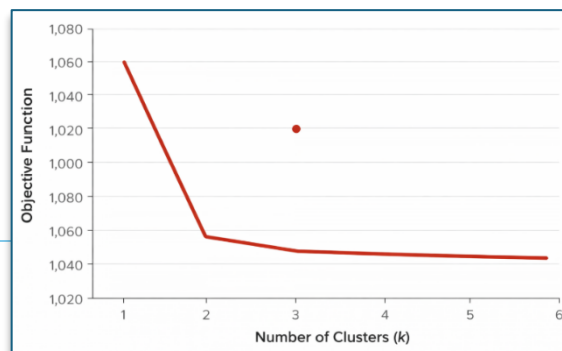
- what **properties** must **centers** $c_1, c_2, \dots, c_k$ have when **distortion** is **minimized**?
 - x_i must be **encoded** by its **nearest center**
 - each **center** must be at the **centroid** of “**owned**” points

Improving a Sub-Optimal Configuration

- what **properties** must be changed for **centers** $c_1, c_2, \dots, c_k$ when **distortion** is **minimized**?
 - change the **encoding** so that x_i is **encoded** by its **nearest center**
 - set each **center** to the **centroid** of “**owned**” points
 - **neither** operation is **repeated** in **succession**
 - the **approach** must be **alternation** (**K-means**)
-
- there are only a **finite number** of ways of partitioning **R records** into **k groups**
 - so, there are **only a finite number** of possible **configurations** in which **all centers** are the **centroids** of the **points** they **own**
 - if the **configuration changes** on an **iteration**, it must have **improved** the **distortion**

	- therefore, each time the configuration changes, it must move to a configuration it has never been to before - if it tried to go on forever, it would eventually run out of configurations
K-Means Algorithm	- decide on a value for K - initialize the K cluster centers randomly if necessary - decide the class memberships of the N objects by assigning them to the nearest cluster centroids (aka the center of gravity or mean) reestimate the K cluster centers, by assuming the memberships found above are correct $\hat{\mu}_k = \left(\frac{1}{c_k} \right) \sum_{i \in C_k} x_i$ - if none of the N objects changed membership in the last iteration, exit - otherwise, repeat the process starting from the membership assignment step
Local Minima and Initialization	
Optimal Solution	the optimal solution is not necessarily reached can you think of a configuration that converges to a state that does not have the minimum distortion?
Trying to Find Good Optima	choose a logical starting point - do many $K - \text{means}$ runs from different start locations

	“neat trick” - place first center on top of randomly chosen datapoint - place second center on data point that is as far away as possible from the first placement - place the j^{th} center on the datapoint that is as far away as possible from the closest of centers 1 through $j - 1$
Choosing K	
Choosing K According to Distortion	- purely choosing according to distortion typically does not sufficiently solve for K – <i>means</i> and/or clustering larger K values lead to less distortion but is not conducive to determining clustering groups
Choosing the Number of Centers	the most common approach is to try to find the solution that minimizes the Schwarz Criterion (also related to the BIC) $Distortion + \lambda(\# parameters) \log R$ m = # dimensions k = # centers R = # records $= Distortion + \lambda m k \log R$
Plot Objective Function Values of $K = 1$ to 6	- we can plot the objective function values for $k = 1$ to 6 - the abrupt change at $k = 2$ is highly suggestive of two clusters in the data - this technique for determining the number of clusters is known as “knee finding” or “elbow finding”



Applications

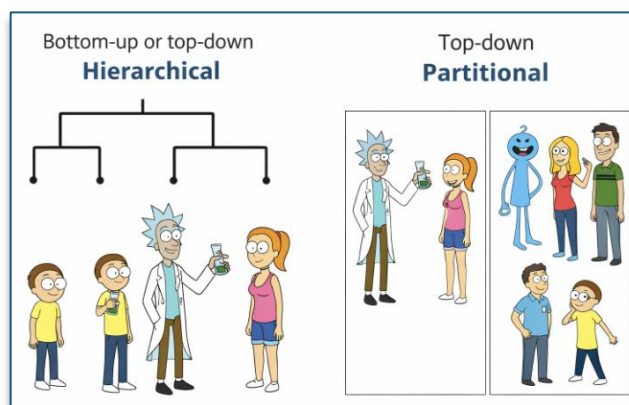
**Common Uses
of K-Means**

- often used as an **exploratory data analysis** tool
- in **one dimension**, a good way to **quantize** real-valued **variables** into **k** non-uniform **buckets**
- used on **acoustic** data, in **speech understanding**, to convert **waveforms** into one of **k** **categories** (known as **vector quantization**)
- also used for **choosing color palettes** on old-fashioned **graphical display** devices

Hierarchical Clustering

**Two Types of
Clustering**

- **hierarchical** algorithms
 - **create** a hierarchical **decomposition** of the set of **objects** using some criterion
- **partitioning** algorithms
 - **construct** various **partitions** and then **evaluate** them by some criterion



ChatGPT 5.4

Hierarchical Clustering (How To)

- **bottom-up** (agglomerative)
 - starting with **each item** in its **own cluster**, find the **best pair** to merge into a **new cluster**
 - **repeat** until all **clusters** have “**fused**” together

Agglomerating Clusters

Distance Matrix

- contains the **distances** between **every pair** of objects in a **database**



	0	3	2	7	8
		0	4	6	8
			0	8	8
				0	3
					0

similarity
assessment:
greater value =
greater distance
(less similar)

$$D(\text{boy}, \text{scientist}) = 3$$

$$D(\text{man}, \text{warrior}) = 3$$

Bottom Up (Agglomerative)

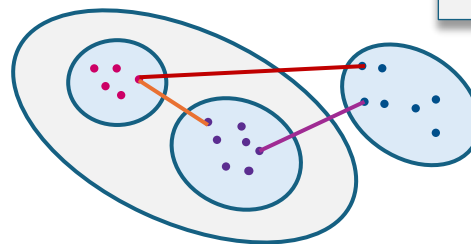
- start with **each item** in its **own cluster**
- find the **best pair to merge** into a new cluster
- **repeat** until **all clusters fuse together**



Linking Options

Computing Distance Between Clusters

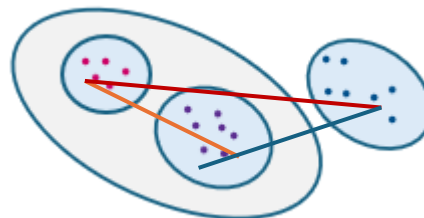
- **single link**
 - cluster **distance** = distance of **two closest** members of **each class**



optimal for long,
skinny clusters

the shortest measured
distance corresponds to
the most similarity and
they would be merged

- **complete link**
 - cluster **distance** = distance of the **two farthest** members



optimal for tight
clusters

- **average link**
 - cluster **distance** =
average distance of all pairs

most used: most
robust to noise